



**University of Stuttgart**  
Germany

# **Systems Design**

**Ilche Georgievski**  
ilche.georgievski@iaas.uni-stuttgart.de

**2026**  
Summer

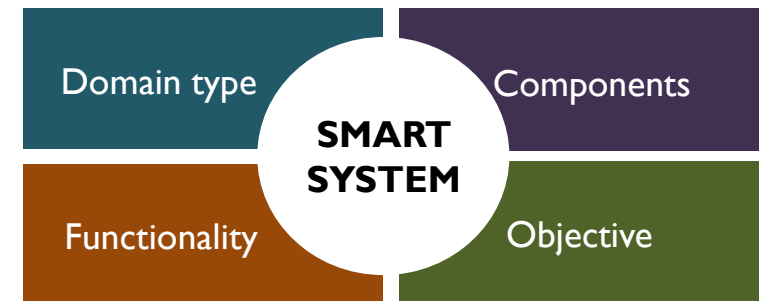
# **SYSTEM DEVELOPMENT PROCESS**

# Phases

- System introduction
- System analysis
- System architecture design
- System implementation

# System introduction

- Project scope
  - Background information
  - Problem statement
- Project goals



# System analysis

- What do the users need and want from the system?
- Deliverable is a list of requirements that will fulfil the system goals
- Two tasks
  - Identify and express system requirements
  - Prioritise system requirements

# Identify and express system requirements

- System users are primary source of requirements
- Types of requirements
  - Functional requirements
  - Non-functional requirements
  - Domain-oriented requirements [1]
  - User-related requirements [2]
- Ways of expression
  - Use cases ([here](#))
  - User stories ([here](#))

[1] Georgievski, I. (2023) Software Development Lifecycle for Engineering AI Planning Systems. In International Conference on Software Technologies, pages 751–760.

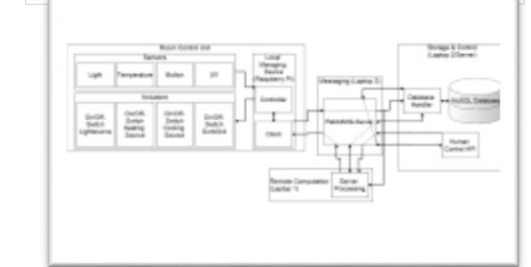
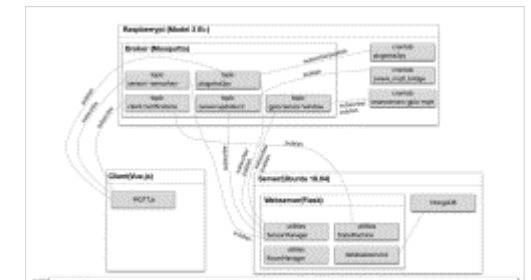
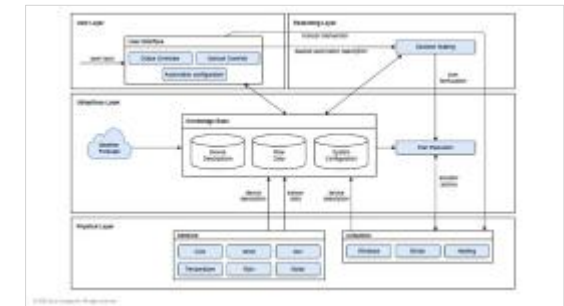
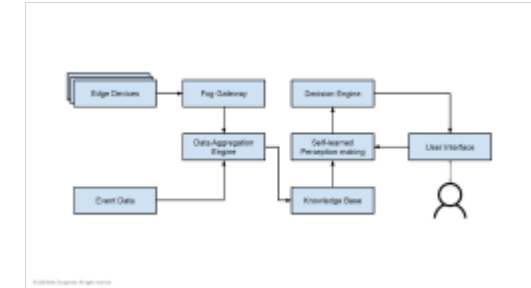
[2] Aiello, M., Fiorini, L., and Georgievski, I. (2021) Software Engineering Smart Energy Systems. Handbook of Smart Energy Systems, pages 1–21.

# Prioritise system requirements

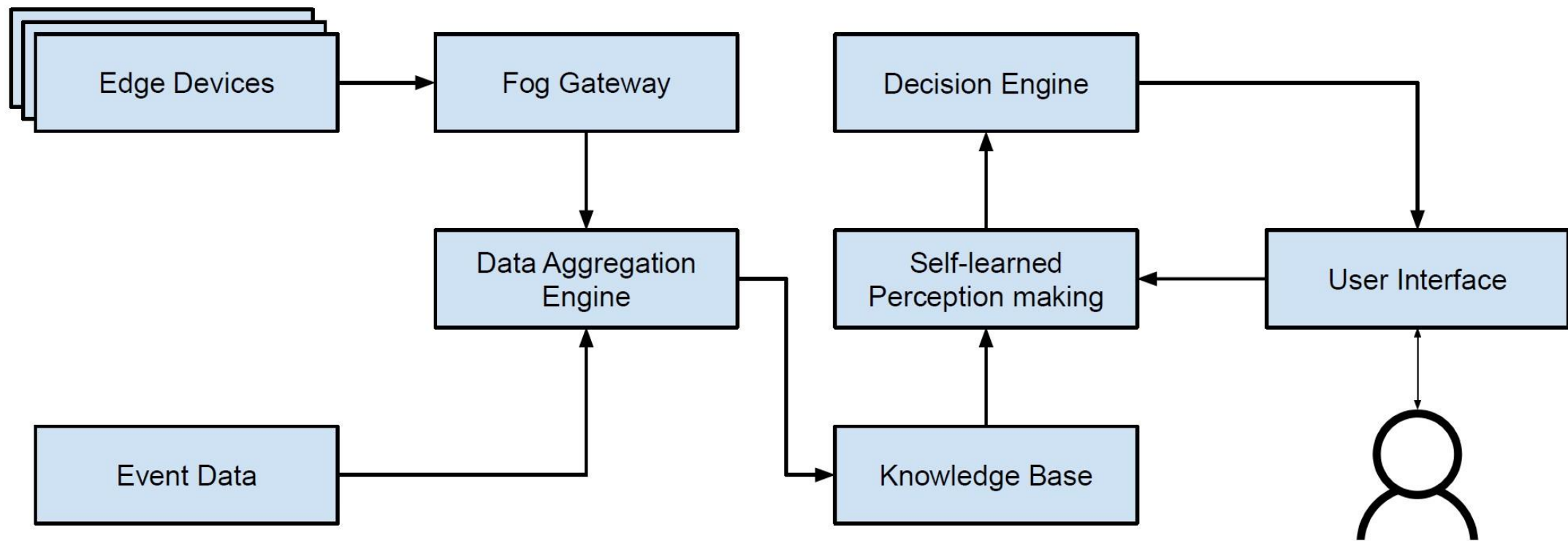
- Mandatory requirements
- Desirable requirements

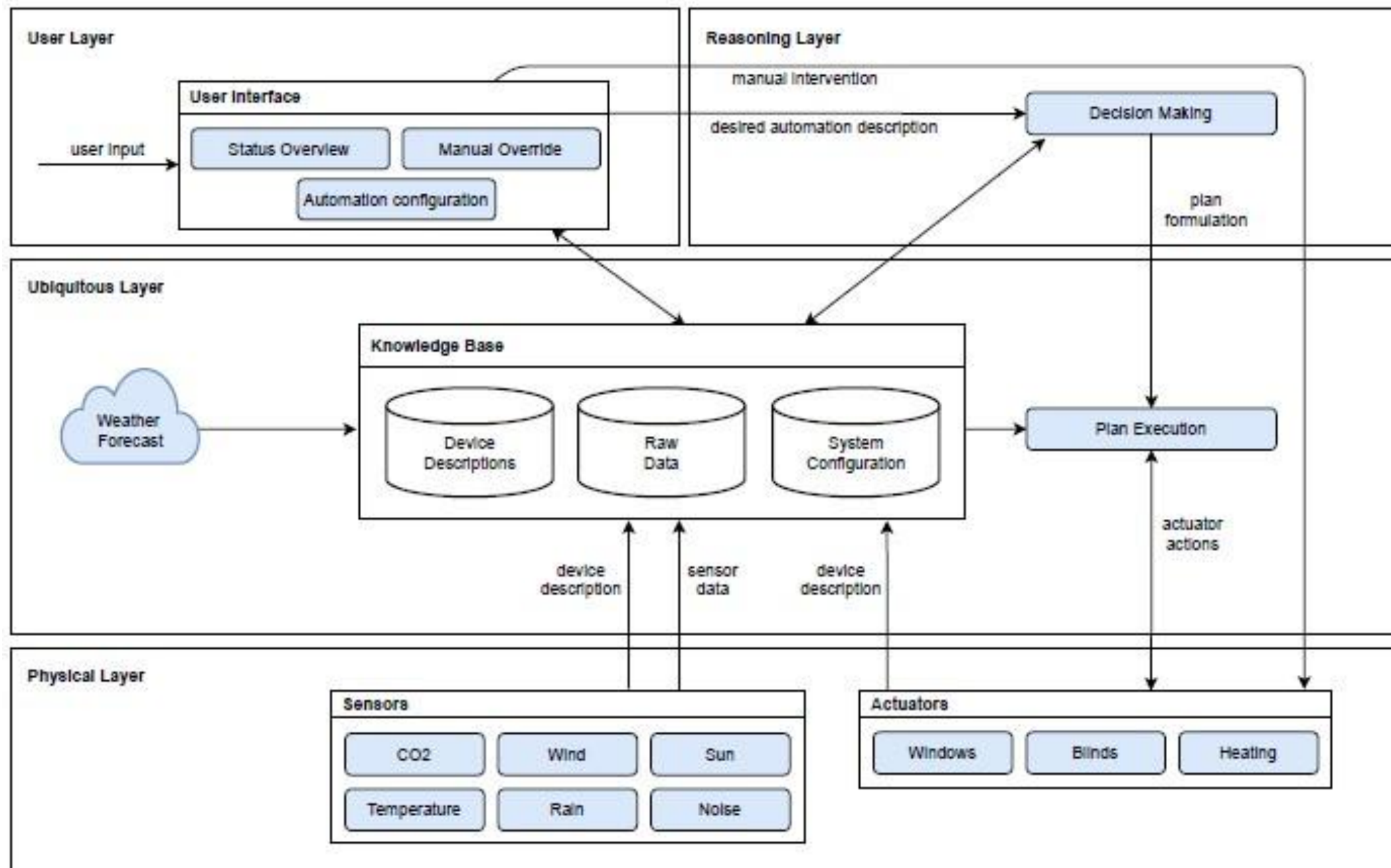
# System architecture design

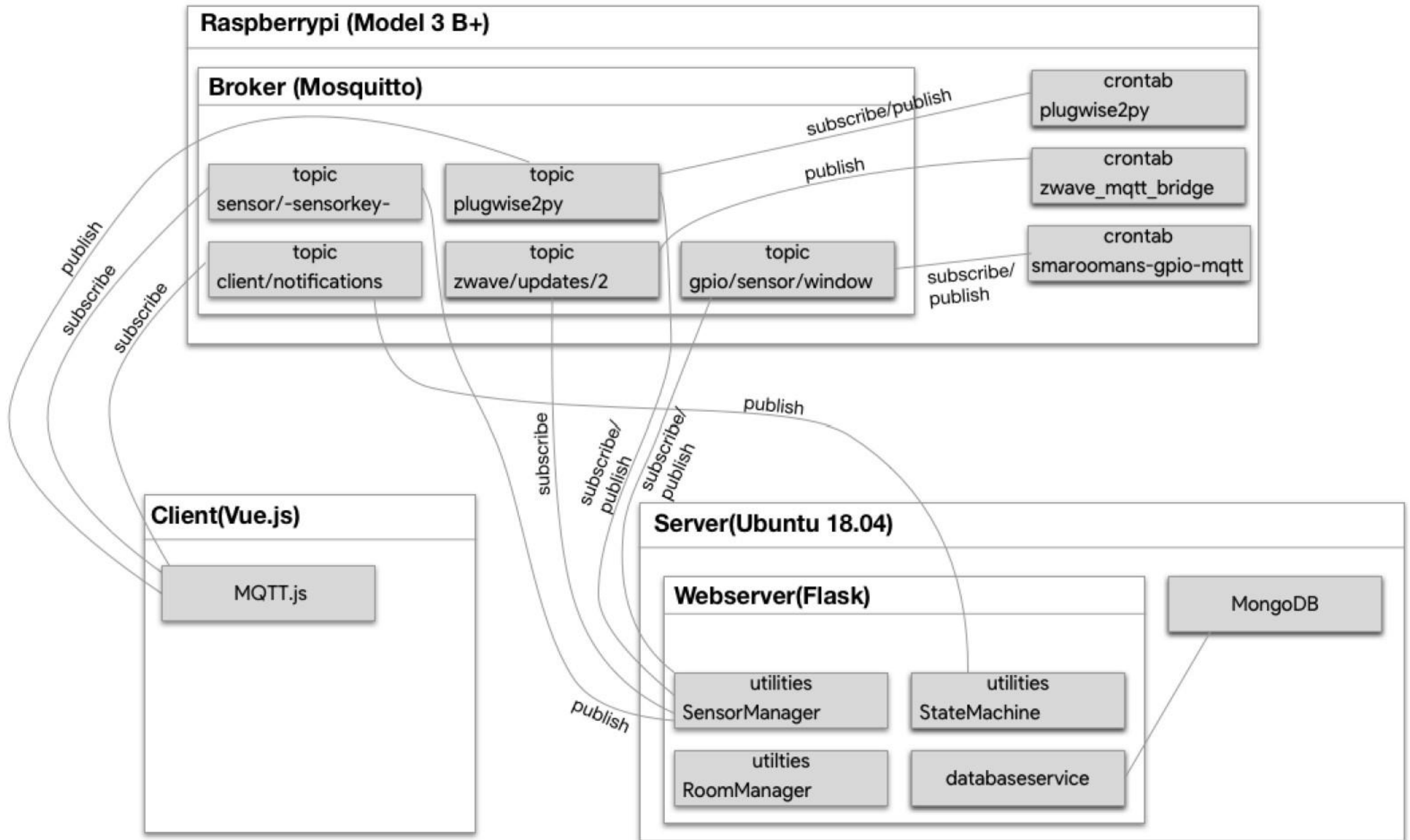
- What a system is or does
- Not how a system is technically implemented
- Implementation independent
- Tasks
  - Identify system components
  - Connect system components and put them in layers
  - Model the system architecture design

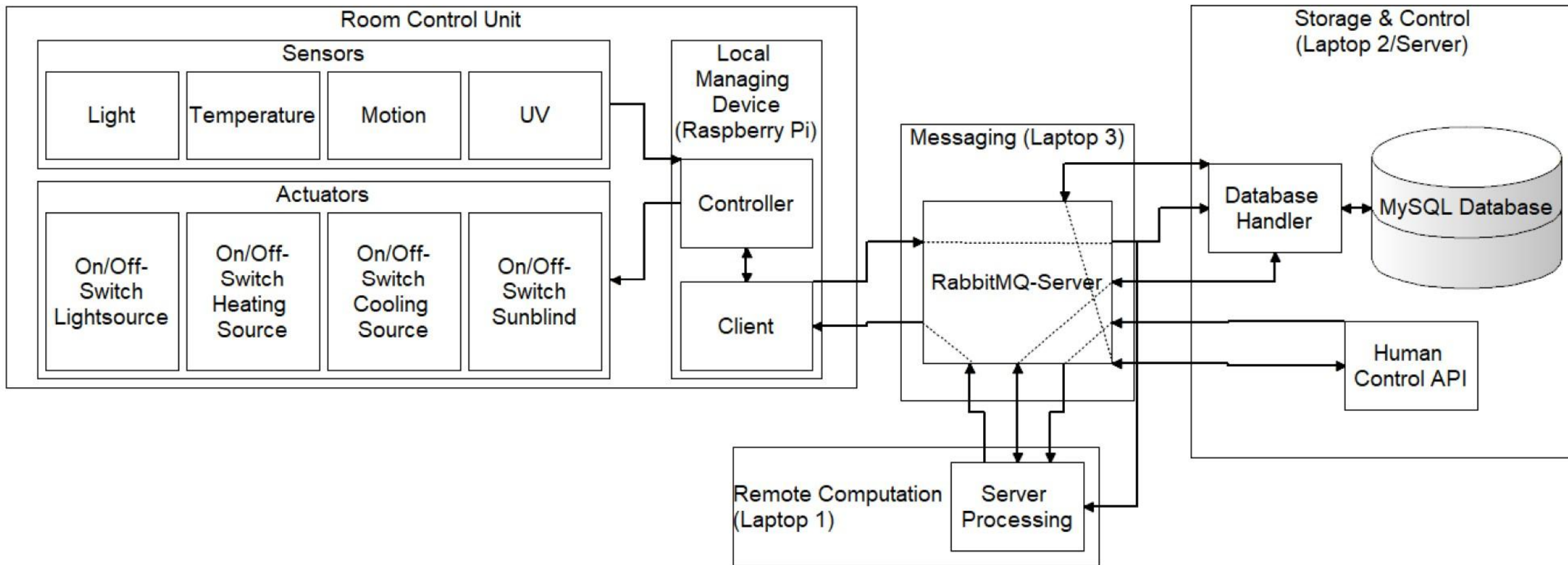












# Identify components

- Component
  - Specific capability/functionality
  - Specific task
  - Group of related functions
- Components loosely coupled
- Components functionally cohesive

# Separation of Concerns (SoC)

- Organising a system into distinct components
  - Each with a specific concern
- Important in smart systems
  - Modularity
  - Scalability and flexibility
  - Improved collaborations
  - Simplifies complexity
  - Better fault tolerance
  - ...

# Connect and layer

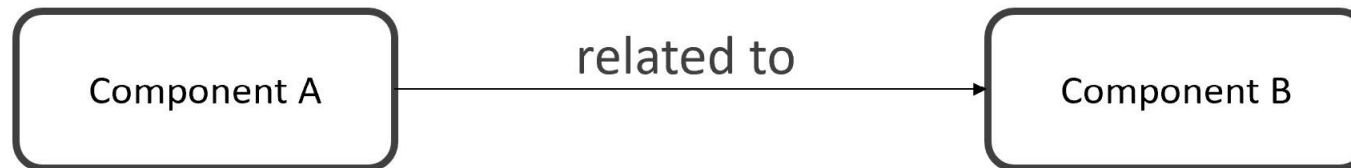
- Presentation layer
- Reasoning layer
- Ubiquitous layer
- Physical layer

Layers of a typical application:

- Presentation layer
- Application logic layer
- Data layer

# Modelling

- Data flow diagram
  - Tutorial [here](#)
- UML Component diagram
  - Tutorial [here](#)
- Simple box-and-line diagram





# System implementation

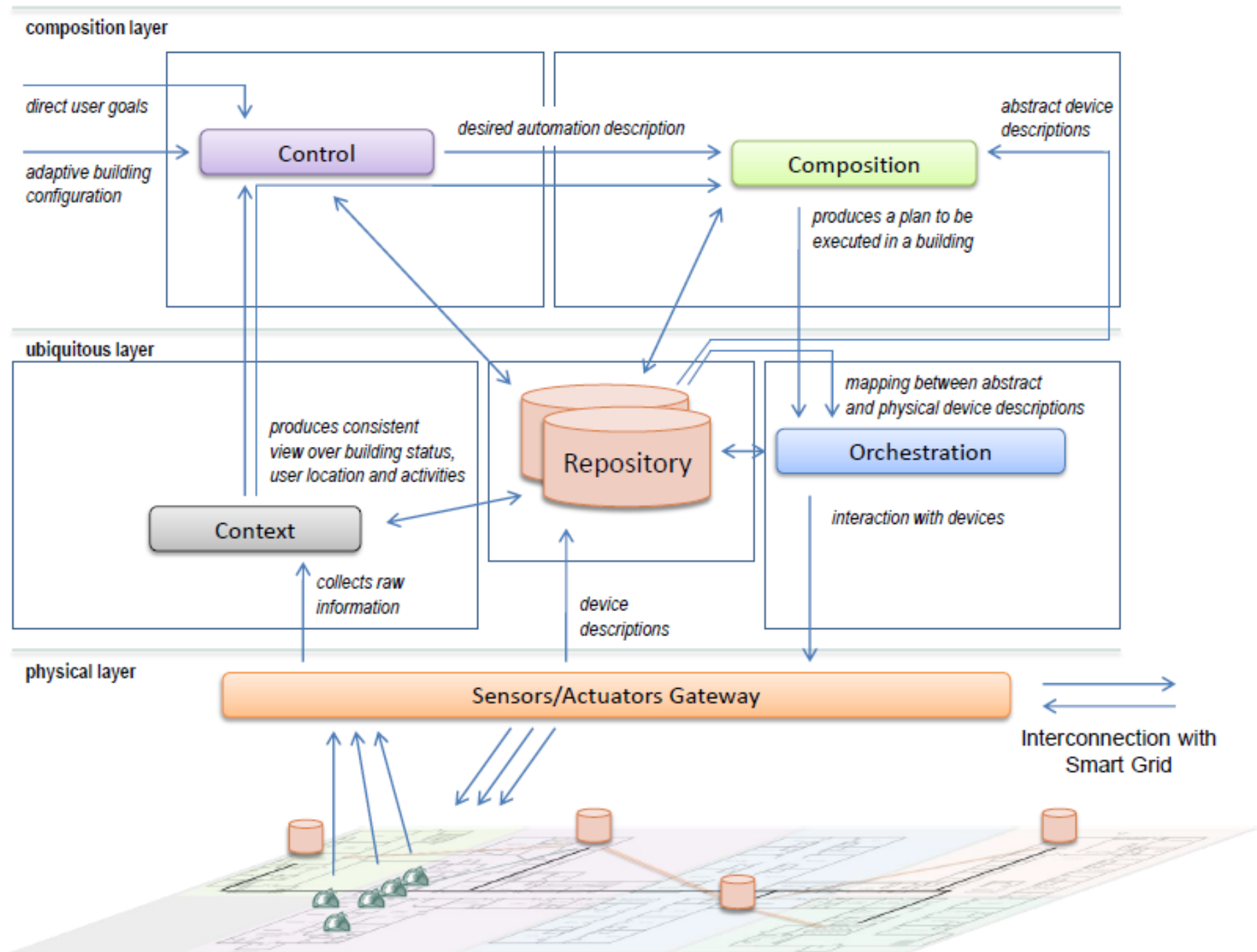
- Implement each system component
- Construct the whole system and put it into operation
- Test
  - System components individually tested
  - Complete system tested
- Tools and methods in subsequent practicals

# Project proposal/Report

- Project information
- System focus
- Project requirements
- System architecture design
- System implementation

– Give a link to code repository (e.g., GitHub, Bitbucket, etc.)

# **EXAMPLES OF SYSTEM ARCHITECTURE DESIGNS FOR SMART SYSTEMS**



Degeler, V., Gonzalez, L. I. L., Leva, M., Shrubsole, P., Bonomi, S., Amft, O., & Lazovik, A. (2013) Service-oriented architecture for smart environments. In IEEE International Conference on Service-Oriented Computing and Applications, pp. 99-104.

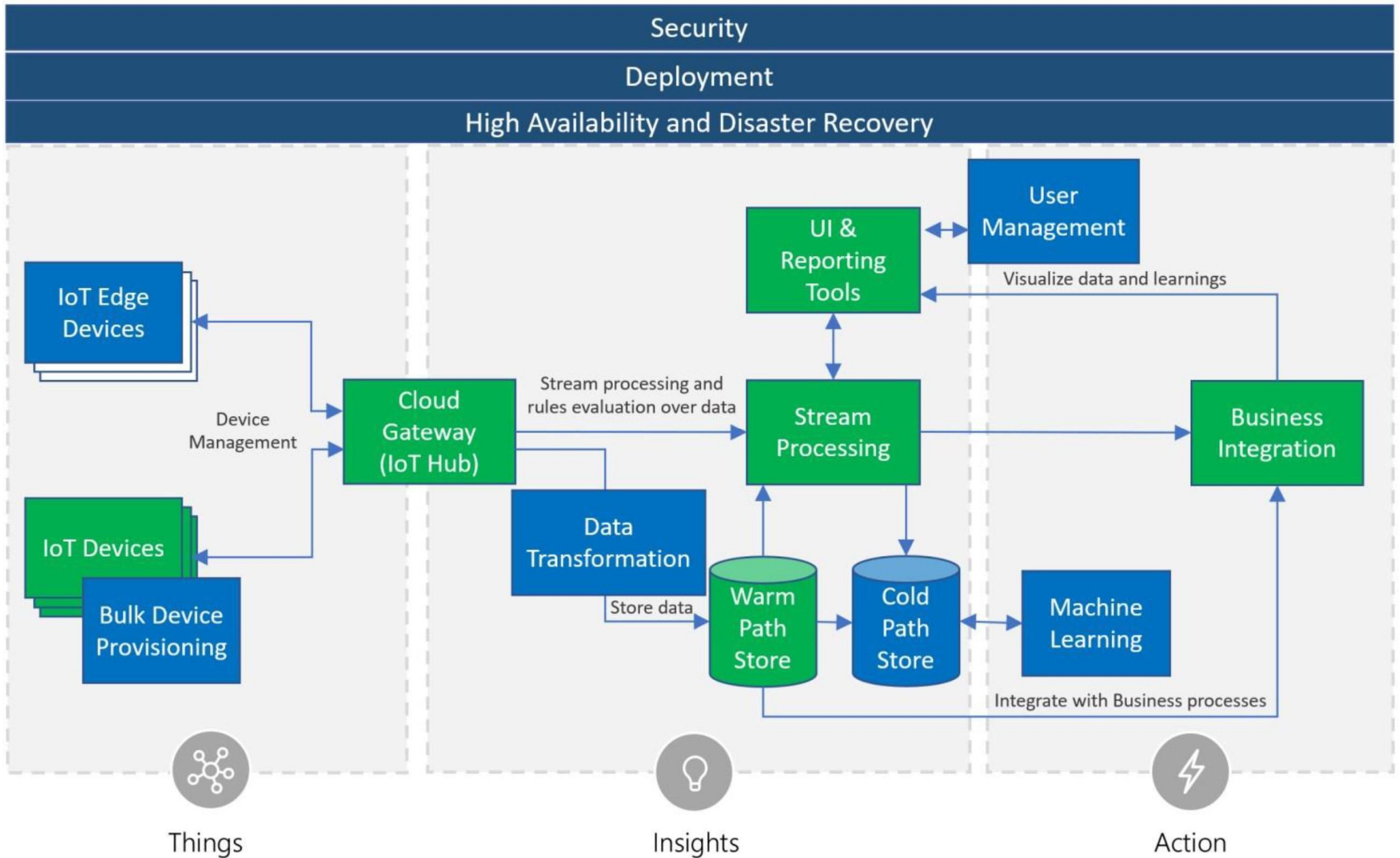
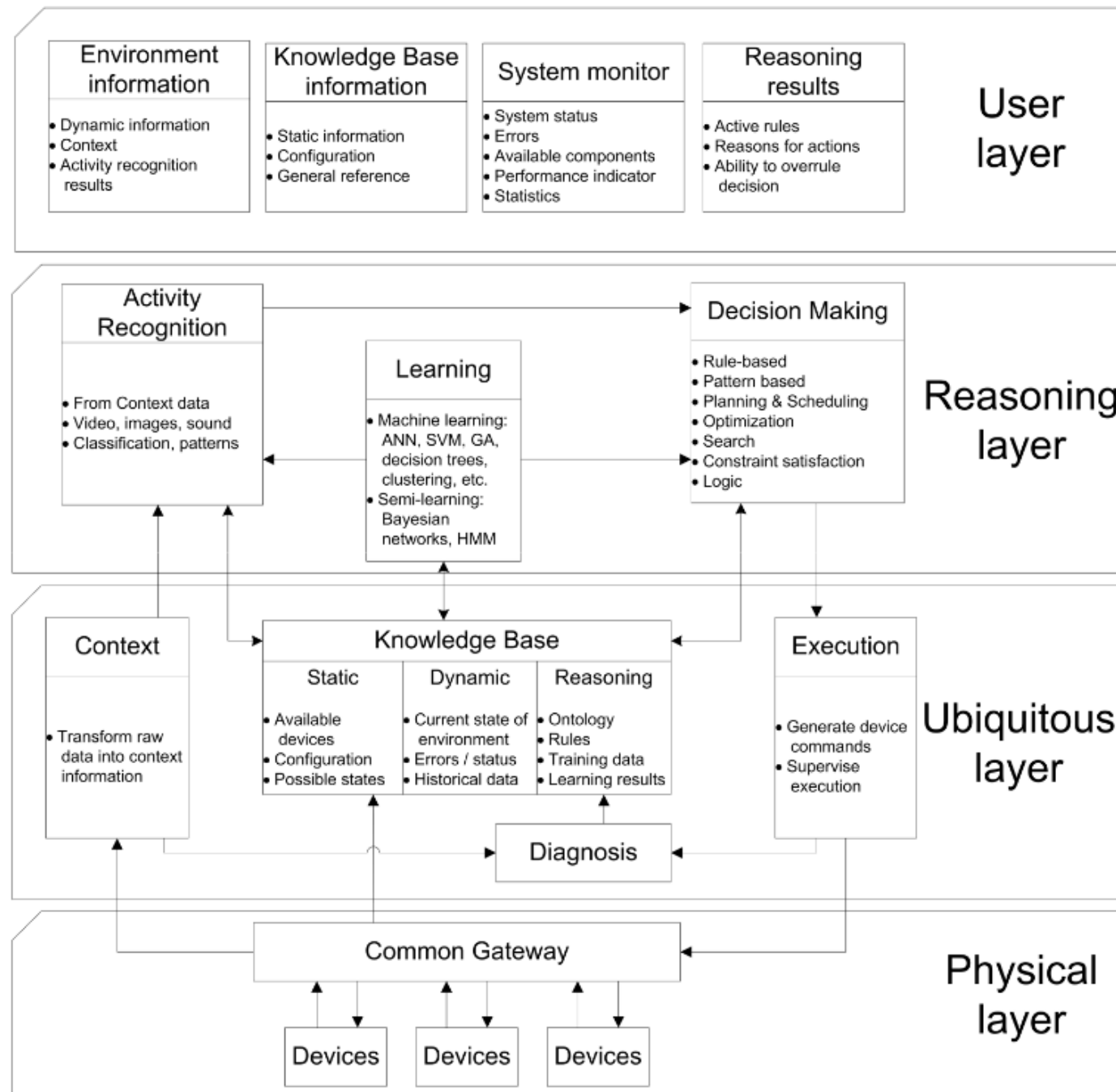


Figure 1: Smart System Architecture Pattern



Degeler, V. and Lazovik, A. (2013) *Architecture pattern for context-aware smart environments*. In *Creating Personal, Social and Urban Awareness through Pervasive Computing*. IGI Global.

# **SYSTEM DESIGN DECISIONS**

# Examples of *what* questions

- What kind of non-residential buildings/autonomous ground vehicles is the system for?
- What types of data describe the system?
- What types of data models characterise the system?
- What kinds of metadata are needed for the system?
- What types of data need to be stored and accessed by the system?
- What context describe the system?
- What kinds of user needs are included in the system?
- What types of control are included in the system?
- What kind of interaction the user can have with the system?
- What information will the system show to user?
- What will an edge component do?
- What are the essential capabilities of the system?



# What types of data describe the system?

- Environmental conditions
  - Temperature
  - Humidity
  - CO<sub>2</sub> level
  - ...
- Device status
  - Light on
  - Radiator valve on
  - Window blinds down
  - ...
- ...

# What types of data models characterise the system?

- Telemetry
- Metadata

# What kinds of metadata are needed for the system?

- Device metadata
- Building metadata

# What types of data need to be stored and accessed?

- Dynamic data
  - Raw stream of incoming data from a device
  - Specific values extracted from the raw data
  - Processed data
- Static data

# What kinds of user needs are included in the system?

- Comfortable working conditions per type of user activity
- Healthy working habits
- Save energy
- ...

# What information will the system show to user?

- Live device data
- Analysis results
- Command and control capabilities
- Notifications
- ...

# Examples of *how* questions

- How is the data collected from IoT formatted?
- How are messages specified and stored?
- How is the metadata specified?
- How is static data stored and accessed?
- How will components communicate among each other?
- How to implement automation?

# How is the metadata specified?

- Device metadata
  - Identifier
  - Class or type
  - Model
  - Hardware serial number
  - ...
- Building metadata
  - Floors
  - Rooms
  - Types of rooms
  - Device location
  - ...



# How is the data collected from IoT formatted?

- Message
  - Type ID
  - Instance ID
  - Timestamp
  - Value

# How to implement automation?

- AI planning
  - Planning domain model
  - Planning problem instance
  - Planner

